



Minmax & alpha-Beta

عملي مشترك

محتوى مجاني غير مخصص للبيع التجاري

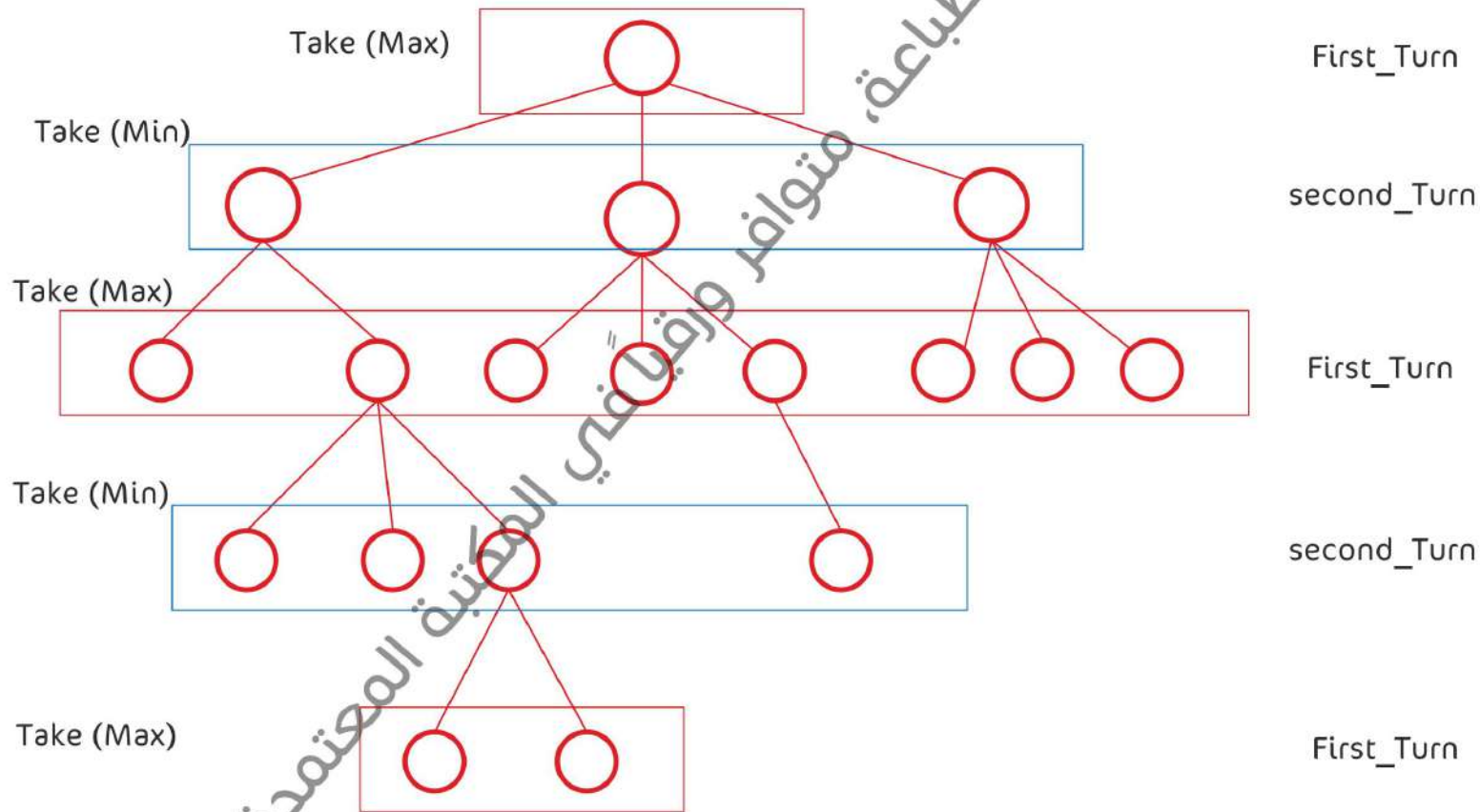
خوارزميات البحث الذكية

20/12/2023

RB Informatics;

بسم الله الرحمن الرحيم

في المحاضرات السابقة تناولنا خوارزميات البحث الذكية للألعاب التي تحتاج لاعب واحد والآن سنبدأ بالخوارزميات للألعاب التي تحتاج لاعبين وأحد أهم هذه الخوارزميات Minmax



■ كيف يستطيع الكمبيوتر bot اللعب بشكل أمثلي دائماً؟!

■ الحقيقة عند اللعب ضد الكمبيوتر عند كل دور لعب لصالح Bot يقوم بالتالي:

1. يفترض أن الخصم (اللاعب الثاني / الانسان) سيلعب بشكل مثالي
2. يجري كل حالات اللعب حيث يختار الأفضل له في دوره والاسوأ له في دور الخصم
3. يستطيع معرفة أفضل نقلة له في حال أن خصمه سيلعب أفضل حركة ممكنة دائماً

هذه الخوارزمية تدعى Minmax

1. لنقوم برسم شجرة لكل حالات اللعب الممكنة
2. نلاحظ أن الـ levels متناوبة (دور الأول - دور الثاني - دور الأول - دور الثاني - ...)
3. الأول يختار الـ option الذي يؤدي إلى أفضل حالة له (Maximize)
4. الثاني يختار الـ option الذي يؤدي إلى أسوأ حالة للأول (Minimize)

ما هو المقياس للحالة الحالية للعبة؟!

عند التفرع نلاحظ أن الأوراق حالات نهائية تنتهي بفوز أحد اللاعبين أو تحقيق some معين لذلك نبدأ من الأوراق ونصعد إلى الأعلى

خطوات الخوارزميات بالتفصيل:

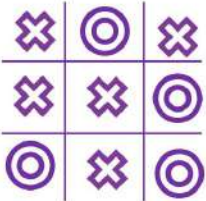
- نبدأ من الأوراق (التي تعتبر حالات نهائية) ونحسب الـ evaluate للـ state حسب دور الشخص الحالي

(على سبيل المثال) حالات اللعب تكون وفق التالي:

حالة تعادل لـ

X&O

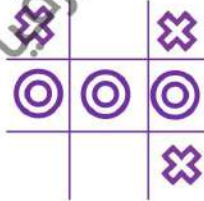
ev = 0



حالة فوز لـ O

ev = $-\infty$

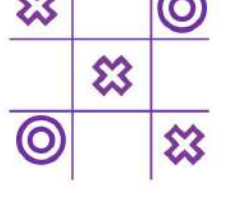
حالة خسارة لـ X

ev = $-\infty$ 

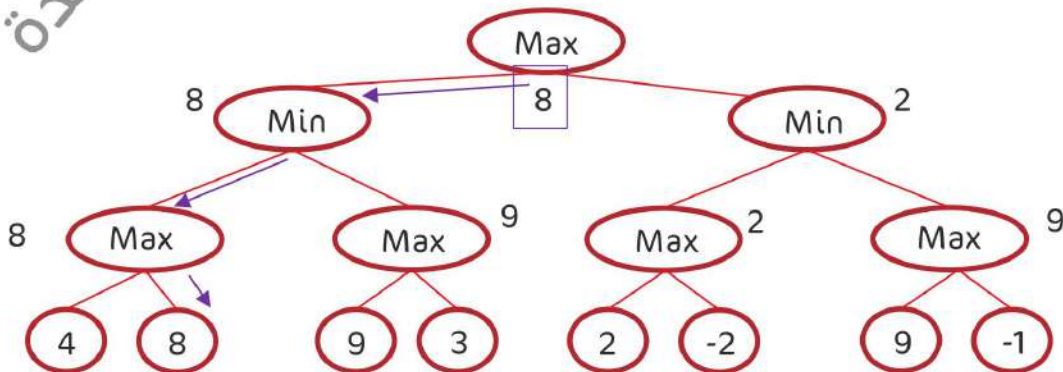
حالة فوز لـ X

ev = $+\infty$

حالة خسارة لـ O

ev = $+\infty$ 

- أما في حالة العقدة الداخلية عندما يكون الدور للاعب الأول نقوم بعمل Max للأبناء (نأخذ أكبر قيمة للأبناء) وعندما يكون الدور للاعب الثاني نقوم بعمل Min للأبناء (نأخذ أصغر قيمة للأبناء) ونعيد القيمة للأعلى (bottom up)، مع الحفاظ على الـ path حتى نستطيع في النهاية معرفة الحركة التالية الأفضل للاعب.



سنقوم بتطبيق الخوارزمية على لعبة Connect_4:

شرح التوابع التي سنستخدمها:

```
public boolean isFinished() {
    return (isWin('x') || isWin('o') || isWithdraw());
}
```

```
public int evaluate(char player) {
    if (isWithdraw()) {
        return 0;
    } else if (isWin(player)) {
        return Integer.MAX_VALUE;
    } else {
        return Integer.MIN_VALUE;
    }
}
```

```
public List<Connect4Game> allNextMoves(char nextPlayer) {
    List<Connect4Game> nextBoards = new LinkedList<>();
    for (int i = 0; i < width; i++) {
        if (topPieceIndex[i] != 0) {
            Connect4Game nextBoard = new Connect4Game(this);
            nextBoard.play(nextPlayer, i);
            nextBoards.add(nextBoard);
        }
    }
    return nextBoards;
}
```

- isFinished() لمعرفة الرقعة نهائية أم لا (العقدة ورقة)
- evaluate() قيمة الرقعة النهائية
- allNextMoves() لمعرفة أبناء العقدة (جميع الحالات التالية)



<https://cutt.ly/5wFaRuYh>

كود البنية كامل



- بما أننا تارةً سنقوم بعمل Max وتارةً Min سنقوم بكتابة تابعين لكل من Maximizing & Minimizing (يمكن دمجهم في تابع واحد).
- بإجراء DFS على شجرة حالات:

- عند الوصول إلى ورقة (حالة نهائية) نحسب الـ evaluate ونعيده
- نمر على جميع الأبناء الحالات التالية ونختار القيمة الأكبر للتابع عند الأبناء في حالة Max والأصغر في حالة Min

```
private void computerPlay() {
    board = maxMove(board).getValue();
}
```

```
private Pair<Integer, Connect4Game> maxMove(Connect4Game b) {
    if (b.isFinished()) {
        int val = b.evaluate('o');
        return new Pair<>(val, b);}
    int val = Integer.MIN_VALUE;
    Connect4Game best = new Connect4Game(b);
    List<Connect4Game> childs = b.allNextMoves('o');
    for (Connect4Game child : childs) {
        Pair<Integer, Connect4Game> option = minMove(child);
        if (val <= option.getKey()) {
            val = option.getKey();
            best = child;
        }
    }
    return new Pair<Integer, Connect4Game>(val, best);
}
```



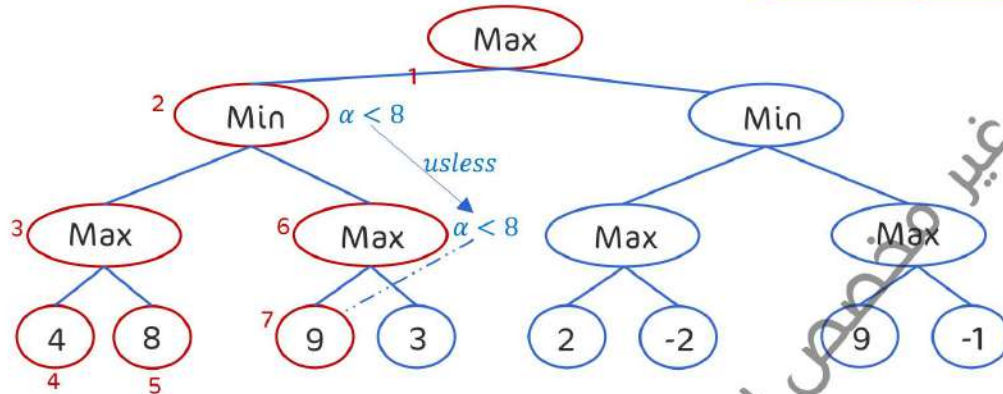
```
private Pair<Integer, Connect4Game> minMove(Connect4Game b) {
    if (b.isFinished()) {
        int val = b.evaluate('x');
        return new Pair<>(val, b);}
    int val = Integer.MAX_VALUE;
    Connect4Game best = new Connect4Game(b);
    List<Connect4Game> childs = b.allNextMoves('x');
    for (Connect4Game child : childs) {
        Pair<Integer, Connect4Game> option = maxMove(child);
        if (val >= option.getKey()) {
            val = option.getKey();
            best = child;
        }
    }
    return new Pair<Integer, Connect4Game>(val, best);
}
```



Alpha Beta pruning

■ نلاحظ أن الخوارزمية Minmax ذات تعقيد عالي جداً أسّي فهل يوجد طريقة لتحسين هذا التعقيد؟!

في الواقع نعم لنكمل هذا المثال:



- عندما نقف عند العقدة رقم 7 في التسلسل DFS نلاحظ أن Min في 2 أصغر أو يساوي 8 وال Max في 6 أكبر أو يساوي 9 فلماذا أقوم بإكمال البحث عن قيمة أكبر لا Max في 6 إذا كان Min لن يستفيد من هذه العقدة في جوابه لذلك أقوم بعمل break وأقطع فروع الشجرة
- هذه الطريقة تعتبر تسريع ل Minmax وتدعى Alpha Beta Pruning
- يكفي أن نحافظ على قيمتين هما α (وهي الحد الأعلى لا Max) و β (الحد الأدنى لا Min)

```
private Pair<Integer, Connect4Game> maxMove(Connect4Game b, int alpha, int beta) {
    if (b.isFinished()) {
        int val = b.evaluate('o');
        return new Pair<>(val, b);
    }
    int val = Integer.MIN_VALUE;
    Connect4Game best = new Connect4Game(b);
    List<Connect4Game> childs = b.allNextMoves('o');
    for (Connect4Game child : childs) {
        Pair<Integer, Connect4Game> option = minMove(child, alpha, beta);
        if (val < option.getKey()) {
            val = option.getKey();
            best = child;
        }
        if (beta <= val)
            break;
        alpha = Integer.max(val, alpha);
    }
    return new Pair<Integer, Connect4Game>(val, best);
}
```

```

private Pair<Integer, Connect4Game> minMove(Connect4Game b, int alpha, int beta) {
    if (b.isFinished()) {
        int val = b.evaluate('x');
        return new Pair<>(val, b); }
    int val = Integer.MAX_VALUE;
    Connect4Game best = new Connect4Game(b);
    List<Connect4Game> childs = b.allNextMoves('x');
    for (Connect4Game child : childs) {
        Pair<Integer, Connect4Game> option = maxMove(child, alpha, beta);
        if (val > option.getKey()) {
            val = option.getKey();
            best = child;
        }
        if (val <= alpha)
            break;
        beta = Integer.min(beta, val);
    }
    return new Pair<Integer, Connect4Game>(val, best);
}

```



انتهت المحاضرة

